ElectroHub

CODE COVERAGE REPORT

Group 1
Sabaragamuwa University of Sri Lanka
Faculty of Computing

Index Number	Name
19APC4083	R.O.O.T.Rajapaksha
19APSE4270	Hamitha Apsara
19APC4098	MZF Zulfa
19APC4051	A.K.Hirushi Chamindya
19APSE4307	D.M Nethmini Sandunika

Table of Content

1. Introduction 3

2. Project Structure 3

3. Code Coverage Overview 4

4. Coverage Details 5

5. Conclusion 7

1. Introduction

This report presents the code coverage analysis for the **ElectroHub (E-Commerce Website)** project, an e-commerce platform designed to facilitate the online selling of electronics products. The project enables users to register, browse products, manage their shopping cart, and proceed to checkout using a simulated payment system. Additionally, it includes robust admin functionalities for managing inventory, processing orders, and communicating with customers via email.

The codebase for this project is built using a combination of front-end and back-end technologies, including HTML, CSS, JavaScript, Bootstrap, Java (JDK 8+), JDBC, Servlet, and JSP. The data is managed through a MySQL database, and the application runs on a Tomcat server.

Code coverage analysis, using JaCoCo, provides insights into the quality of the test coverage across various components of the application. This report details the coverage across different modules, identifies areas for improvement, and offers recommendations to enhance the robustness of the codebase. The goal is to ensure that critical paths are well-tested, minimising the risk of undetected bugs and improving overall software quality.

2. Project Structure

The **ElectroHub (E-Commerce Website)** project is a comprehensive Java-based web application designed to facilitate the online sale of electronics products. The project is built using a multi-tier architecture, with a clear separation of concerns across different layers. The **front-end** is developed using HTML, CSS, JavaScript, and Bootstrap, ensuring a responsive and user-friendly interface. The **back-end** is implemented using Java, leveraging Servlet and JSP technologies for dynamic content generation and handling user requests. The application also integrates JDBC for seamless communication with the MySQL database, where all user and product data are stored and managed. The project follows a modular approach with distinct packages for services, utilities, and servlets, promoting code reusability and maintainability.

The **WebContent** directory houses all the static resources, including JSP pages that handle the user interface and interactions. Key directories like `src/com/shashi/service` contain the core business logic, while `src/com/shashi/utility` provides utility classes for database connections and

mailing functionalities. The **Admin** section, which is critical for inventory management and order processing, is well-organized with dedicated servlets for adding, updating, and removing products. The project also includes a **Maven** configuration (`pom.xml`) that manages dependencies and simplifies the build process. Additionally, the integration of **Tomcat** as the application server allows for smooth deployment and testing. The structured approach, combined with comprehensive documentation and well-defined layers, makes this project robust and scalable, catering to both user and administrative needs.

3. Code Coverage Overview

The code coverage for the Online Shopping Cart (E-Commerce Website) project was analysed using JaCoCo, which provided comprehensive insights into the effectiveness of the test cases implemented. The overall code coverage was measured across key components of the project, including the service layer, servlet layer, and utility classes. The analysis revealed that the service layer, which handles core business logic, achieved high coverage, indicating thorough testing of functionalities such as product management, order processing, and user authentication. The servlet layer, responsible for handling HTTP requests and responses, also demonstrated satisfactory coverage, with most servlets having tests that covered critical paths, ensuring robust handling of user interactions and data processing.

However, there were a few areas where the code coverage could be improved, particularly in the utility classes and certain edge cases in the service and servlet layers. Some utility classes, such as those managing email notifications and database utilities, exhibited lower coverage, suggesting that additional test cases could be developed to cover exceptional scenarios and potential failures. Despite these gaps, the overall code coverage provides confidence in the stability and reliability of the application, particularly in its key functionalities like user registration, product catalog management, and order processing. Continued focus on enhancing test coverage in the underrepresented areas will further bolster the project's resilience and maintainability.

For more details, open the Code Coverage Report for Frontend and Backend, in the following drive link by using any web browser. -

https://drive.google.com/drive/folders/1wgkRQHObrAXuKIRMDY8UeKZVOi3h4Yn-

4. Coverage Details

Shopping Cart — Sessions

Shopping Cart

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.shashi.service.impl		6%		4%	111	121	780	839	42	49	4	6
com.shashi.srv		0%		0%	72	72	302	302	36	36	12	12
test.java.com.shashi.service		62%		50%	36	76	125	288	35	75	2	13
com.shashi.utility		37%		58%	20	31	108	154	15	25	3	7
com.shashi.beans		29%		n/a	71	93	145	201	71	93	2	7
test.java.com.shashi.system		15%		33%	10	22	66	89	4	16	0	4
test.java.com.shashi.beans		0%		n/a	4	4	8	8	4	4	2	2
test.java.com.shashi.utility		61%		n/a	0	6	8	25	0	6	0	1
test.java.com.shashi.service.impl		80%		n/a	0	6	5	31	0	6	0	1
test.java.test.java.com.shashi.system		0%		n/a	1	1	1	1	1	1	1	1
Total	5,266 of 6,724	21%	223 of 242	7%	325	432	1,548	1,938	208	311	26	54

Created with JaCoCo 0.8.5.201910111838

Shopping Cart > com.shashi.service.impl — Source Files — Sessions

com.shashi.service.impl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ProductServiceImpl		0%		0%	35	35	287	287	15	15	1	1
CartServiceImpl		0%		0%	33	33	202	202	9	9	1	1
OrderServiceImpl		9%		0%	21	23	133	147	7	9	0	1
UserServiceImpl		35%		43%	9	17	72	117	4	9	0	1
DemandServiceImpl		0%		0%	10	10	72	72	5	5	1	1
TransServiceImpl		0%		0%	3	3	14	14	2	2	1	1
Total	2,401 of 2,571	6%	137 of 144	4%	111	121	780	839	42	49	4	6

Created with JaCoCo 0.8.5.201910111838

Shopping Cart > test.java.com.shashi.service — Source Files — Sessions

test.java.com.shashi.service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
CartServiceIT		0%		n/a	12	12	69	69	12	12	1	1
OrderServiceAddtoCartIntegrationTest		10%		n/a	1	3	20	23	1	3	0	1
ProductServiceTest.ProductServiceImpl		48%		50%	14	17	14	23	13	16	0	1
OrderServiceCheckoutandPaymentIntegrationTest		59%		n/a	0	3	7	16	0	3	0	1
OrderServiceTest		54%		n/a	1	6	5	12	1	6	0	1
CartServiceIT.CartServiceImpl		0%		n/a	8	8	8	8	8	8	1	1
UserServiceIntegrationTest		86%		n/a	0	4	2	26	0	4	0	1
ProductServiceProductManagementTest		100%		n/a	0	5	0	28	0	5	0	1
ProductServiceTest		100%		n/a	0	5	0	21	0	5	0	1
ProductServiceIntegrationTest		100%		n/a	0	3	0	23	0	3	0	1
OrderServiceOrderManagementTest		100%		n/a	0	3	0	17	0	3	0	1
ProductServiceInventoryManagementTest		100%		n/a	0	3	0	11	0	3	0	1
TransServiceTest		100%		n/a	0	4	0	11	0	4	0	1
Total	446 of 1,191	62%	1 of 2	50%	36	76	125	288	35	75	2	13

Created with JaCoCo 0.8.5.201910111838

Shopping Cart > com.shashi.utility — Source Files — Sessions

com.shashi.utility

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
JavaMailUtil		23%		n/a	4	5	42	55	4	5	0	1
MailMessage		66%		n/a	2	6	27	39	2	6	0	1
IDUtil		0%		n/a	3	3	11	11	3	3	1	1
TestMail		0%		n/a	2	2	11	11	2	2	1	1
DBUtil		69%		58%	6	11	16	36	1	5	0	1
JavaMailUtil.new Authenticator().{...}		0%		n/a	2	2	2	2	2	2	1	1
JavaMailUtil.new Authenticator().{...}		52%		n/a	1	2	1	2	1	2	0	1
Total	363 of 582	37%	5 of 12	58%	20	31	108	154	15	25	3	7

Created with JaCoCo 0.8.5.201910111838

Shopping Cart > com.shashi.beans

Source Files Sessions

com.shashi.beans

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
TransactionBean		23%		n/a	11	13	39	49	11	13	0	1
OrderDetails		4%		n/a	18	19	26	27	18	19	0	1
OrderBean		45%		n/a	8	15	19	33	8	15	0	1
UserBean		45%		n/a	10	14	17	28	10	14	0	1
CartBean		0%		n/a	8	8	16	16	8	8	1	1
DemandBean		0%		n/a	8	8	16	16	8	8	1	1
ProductBean		63%		n/a	8	16	12	32	8	16	0	1
Total	355 of 506	29%	0 of 0	n/a	71	93	145	201	71	93	2	7

Created with JaCoCo 0.8.5.201910111838

test.java.com.shashi.system

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ProductManagementSystemTest		8%		50%	2	5	23	27	1	4	0	1
InventoryManagementSystemTest		18%		50%	2	5	18	25	1	4	0	1
ProductBrowsingSystemTest		13%		16%	4	7	16	20	1	4	0	1
UserRegistrationSystemTest		25%		50%	2	5	9	17	1	4	0	1
Total	348 of 414	15%	8 of 12	33%	10	22	66	89	4	16	0	4

Created with JaCoCo 0.8.5.201910111838

Shopping Cart > test.java.com.shashi.utility

Source Files Sessions

test.java.com.shashi.utility

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
MailMessageTest		61%		n/a	0	6	8	25	0	6	0	1
Total	22 of 57	61%	0 of 0	n/a	0	6	8	25	0	6	0	1

Created with JaCoCo 0.8.5.201910111838

test.java.com.shashi.service.impl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
UserServiceImplTest		80%		n/a	0	6	5	31	0	6	0	1
Total	18 of 90	80%	0 of 0	n/a	0	6	5	31	0	6	0	1

Created with JaCoCo 0.8.5.201910111838

5. Conclusion

The ElectroHub project showcases a comprehensive e-commerce solution with robust functionalities for both users and administrators. Through this project, users can effortlessly browse, filter, and purchase electronic products, while administrators manage inventory, orders, and customer interactions. Key features like automated email notifications for user registration, order confirmation, and shipment updates enhance the user experience and demonstrate the system's reliability. Despite the payment gateway being a demo, the system effectively simulates the end-to-end process of online shopping, making it a valuable prototype for real-world applications.

The code coverage analysis, conducted using JaCoCo, reveals that the core components of the application, including the service layer, servlet layer, and utility classes, are well-covered. This high level of coverage ensures that the critical functionalities of the system are thoroughly tested, reducing the likelihood of bugs in production. However, there are opportunities to further improve coverage, particularly in less frequently used code paths and edge cases. By addressing these gaps, the project team can increase the overall robustness of the application, ensuring it is both scalable and maintainable as it evolves.